Project - High Level Design on

CI/CD Pipeline for Nodejs Retail-app

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Arsh Patidar	EN22CS301204
2	Anuj Singh Rathore	EN22CS301166
3	Anurag Didolkar	EN22CS301169
4	Avani Gupta	EN22CS301236
5	Amit Patidar	EN22CS301114
6	Aniket Kushwah	EN22CS301124

Group Name: 09D2

Project Number: DO-21

Industry Mentor Name: Mr. Vaibhav

University Mentor Name:

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **CI/CD for Node.js Retail Application** project is a complete DevOps implementation that automates the software development lifecycle from code commit to production deployment. The system is designed to demonstrate modern DevOps practices by integrating source control, continuous integration, containerization, artifact management, and container orchestration.

The application itself is a retail service built using Node.js, exposing APIs for product management and order processing. The DevOps pipeline ensures that every code change is automatically tested, packaged into a Docker container, stored in a registry, and deployed to a Kubernetes cluster.

This project emphasizes:

- Automation of build, test, and deployment workflows
- High availability and scalability using Kubernetes
- Consistent environments via containerization
- Faster and reliable delivery cycles

1.1 Scope of the Document

This document provides a high-level overview of the system architecture, components, workflows, and design considerations. It focuses on:

- CI/CD pipeline design
- Application deployment strategy
- Data and system interactions
- Non-functional requirements

It does not include low-level implementation details or code.

1.2 Intended Audience

- DevOps Engineers responsible for pipeline implementation
- Developers contributing to the Node.js application
- Cloud Engineers managing Kubernetes infrastructure
- QA Engineers validating automated testing
- Architects reviewing system scalability and reliability

1.3 System Overview

The system automates the end-to-end lifecycle:

1. Developer commits code to Git repository

2. CI/CD pipeline is triggered automatically
3. Application is built and tested
4. Docker image is created and pushed to registry
5. Kubernetes deploys the updated application
6. End users access the deployed retail service

The system ensures minimal manual intervention, improved code quality, and continuous delivery.

2. System Design

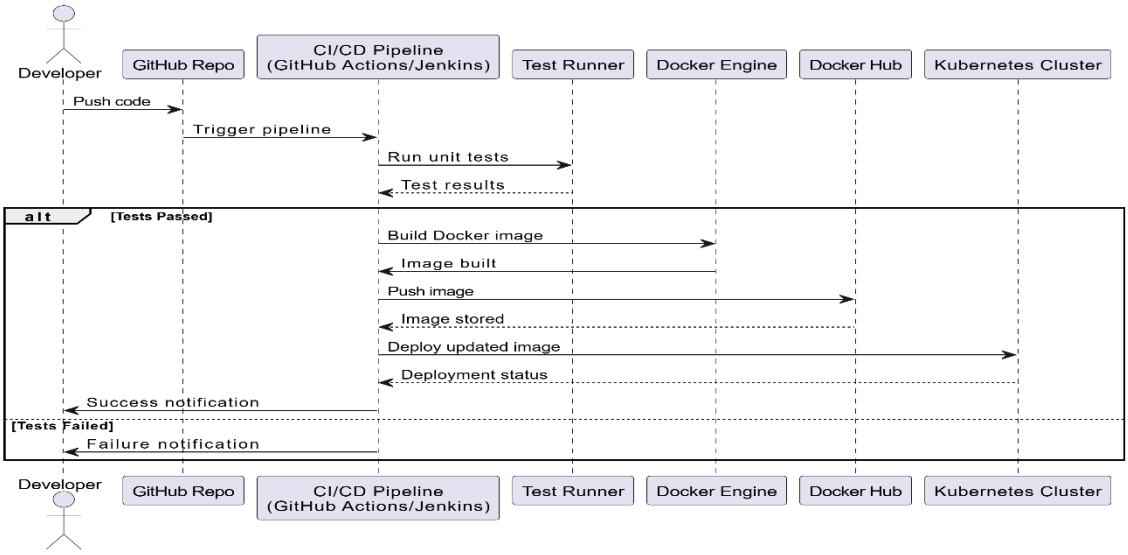
The application follows a modular and scalable architecture:

- Backend Framework: Node.js with Express
- Architecture Style: Monolithic (microservice-ready)
- Deployment Unit: Docker container
- Environment: Kubernetes cluster

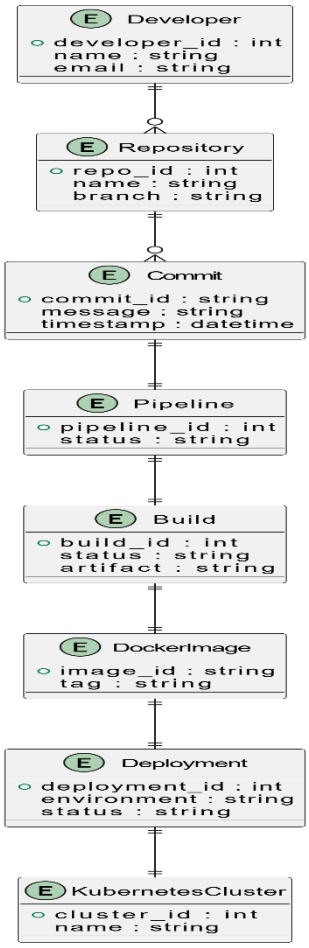
Design Principles:

- Loose coupling between components
- Stateless service design
- API-first development approach
- Container-first deployment strategy

Sequence Diagram:

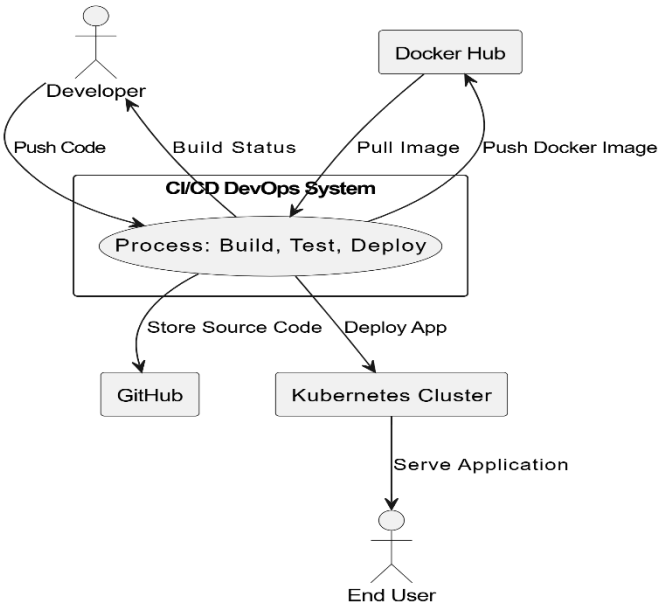


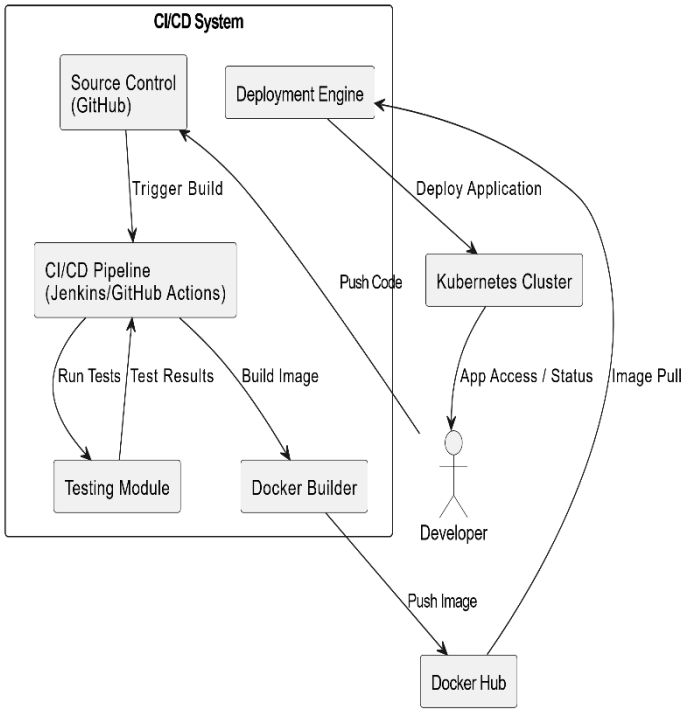
ER Diagram:



Data Flow Diagram:

Level 0:





2.2 Process Flow

The CI/CD pipeline executes the following steps: Code commit triggers pipeline

1. Source code is checked out
2. Dependencies are installed
3. Static code analysis is performed
4. Unit tests are executed
5. Build process creates application artifact
6. Docker image is built
7. Image is pushed to Docker Hub
8. Kubernetes deployment is updated
9. Health checks validate deployment
10. Notifications are sent to developers

2.3 Information Flow

- Developer → Git: Source code push
- Git → CI/CD: Webhook trigger
- CI/CD → Test Module: Code validation
- CI/CD → Docker: Image build
- Docker → Registry: Image storage
- Registry → Kubernetes: Image pull
- Kubernetes → User: Application response

This ensures a seamless flow from development to production.

2.4 Components Design

1. Source Control

- Git-based repository (branching strategy supported)
- Maintains version history and collaboration

2. CI/CD Pipeline

- Automates build, test, and deployment
- Supports parallel job execution
- Provides logs and status reporting

3. Docker Engine

- Builds container images
- Ensures consistency across environments

4. Container Registry

- Stores versioned Docker images
- Enables rollback using image tags

5. Kubernetes Cluster

- Manages container orchestration
- Handles scaling, load balancing, and failover

6. Networking

- Kubernetes Services expose applications
- Ingress enables external access

- **2.5 Key Design Considerations**
- Horizontal scaling using Kubernetes replicas
- Auto-scaling based on CPU/memory

Reliability

- Self-healing containers
- Rolling updates with zero downtime

Maintainability

- Modular pipeline stages
- Version-controlled infrastructure

Security

- Secrets management
- Secure image repositories
- Role-based access control

Performance

- Optimized Docker builds
- Efficient API design

2.6 API Catalogue

Product APIs

- GET /products – Retrieve all products
- GET /products/{id} – Retrieve specific product

Order APIs

- POST /orders – Create order
- GET /orders/{id} – Get order details

System APIs

- GET /health – Health check
- GET /metrics – Monitoring metrics

3. Data Design

3.1 Data Model

Core entities include:

- **User**: Stores customer details

- **Product**: Contains product catalog
- **Order**: Tracks purchases
- **Cart**: Temporary user selections

Relationships:

- One user can have multiple orders
- Orders contain multiple products

.

3.2 Data Access Mechanism

- RESTful API communication
- Database access via ORM/ODM
- Connection pooling for efficiency

3.3 Data Retention Policies

- Application logs retained for debugging
- Transactional data stored long-term
- Automated backups for recovery

3.4 Data Migration

- Schema evolution handled via migrations
- Backward compatibility maintained
- Zero-downtime deployments supported

4. Interfaces

External Interfaces

- REST APIs over HTTPS
- Docker registry APIs
- Kubernetes API server

Internal Interfaces

- Pipeline stage communication
- Container-to-container communication

5. State and Session Management

- Stateless application design
- Session handled via:
  - JWT tokens
  - Token-based authentication
- Kubernetes manages scaling without session loss

6. Caching

- Redis (optional) for caching
- API-level caching to reduce latency
- Improves performance and scalability

7. Non-Functional Requirements

7.1 Security Aspects

- Secure communication using HTTPS
- Secrets stored in Kubernetes Secrets
- Image vulnerability scanning
- Authentication and authorization controls

7.2 Performance Aspects

- Low response time APIs
- Efficient container startup
- Load balancing via Kubernetes
- Auto-scaling under high traffic

8. References

- Official Docker Documentation
- Kubernetes Documentation
- Node.js Documentation
- CI/CD Tool Documentation (GitHub Actions/Jenkins)
- DevOps Best Practices